

ICPC Bootcamp Day 4 Summary

Based on `day4_slides.pptx`

Session Summary

Day 4 focuses on graph algorithms. The opening material reviews graph traversal: systematically visiting vertices and edges to discover reachability, components, cycles, paths, or orderings. The slides assume adjacency lists as the normal implementation because most contest graphs are sparse and neighbor iteration is central to traversal.

Depth first search explores one path as deeply as possible before backtracking. It is commonly implemented with recursion, using the function call stack as the implicit stack of active vertices. DFS runs in $\mathcal{O}(V + E)$ time and is a natural tool for reachability, connected components, cycle detection, and topological sorting. The deck also introduces DFS edge intuition, including back edges as evidence of cycles.

Breadth first search explores layer by layer using a FIFO queue. Like DFS, it runs in $\mathcal{O}(V + E)$, but its key contest property is different: in an unweighted graph, the first time BFS reaches a vertex, it has found a shortest path in number of edges. The deck contains an extended BFS trace showing the frontier, the queue, discovered vertices, and distance values as the search expands outward. The DFS/BFS comparison slide summarizes the rule of thumb: use BFS for unweighted shortest paths and minimum moves; use DFS when connectivity, components, cycles, or recursive structure are the focus.

Flood fill is presented as graph traversal on an implicit grid. A maze, map, or image can be treated as a graph where each cell has predictable neighbors such as up, down, left, and right. Instead of constructing a large adjacency list, students can use direction arrays and boundary checks to generate neighbors on demand. This keeps the implementation short and avoids unnecessary memory use.

The second topic is minimum spanning tree. The *Lost Map* case study is modeled as a complete undirected weighted graph: villages are vertices, distances are edge weights, and the task is to choose roads that connect all villages with minimum total length. An MST of a connected undirected weighted graph selects $V - 1$ edges that keep the graph connected while minimizing total weight.

Kruskal's algorithm solves MST by sorting all edges in nondecreasing weight and greedily adding an edge if it does not create a cycle. Union-find disjoint sets provide the cycle check by asking whether the two endpoints are already in the same component. The deck also contrasts Kruskal with Prim: Kruskal processes edges globally and fits edge-list inputs, while Prim grows from vertices and often fits adjacency-list or dense-graph contexts.

The final topic is single-source shortest paths. The slides define relaxation: if going from u to v improves `dist[v]`, update it. BFS handles unweighted shortest paths because every edge has equal cost. Dijkstra's algorithm handles weighted graphs with nonnegative edge weights by repeatedly processing the unvisited vertex with smallest known distance using a priority queue. The deck closes with a step-by-step Dijkstra trace and warns that negative edge weights break Dijkstra's correctness assumption.

Key Takeaways

- DFS and BFS have the same asymptotic traversal cost but solve different contest tasks.
- Flood fill is BFS or DFS on a grid without explicitly building the graph.
- MST problems ask for minimum-cost connectivity, not shortest paths from one source.
- Kruskal combines sorting with union-find; Dijkstra combines relaxation with a priority queue and requires nonnegative weights.

CP4 Reading Connections

- CP4 Section 2.4.1, *Graph*, for adjacency matrix, adjacency list, and edge list representations.
- CP4 Section 4.2, *Graph Traversal*, especially Sections 4.2.2, 4.2.3, and 4.2.5.
- CP4 Section 4.3, *Minimum Spanning Tree*, especially Sections 4.3.1 and 4.3.2; also review Section 2.4.2 for union-find.
- CP4 Section 4.4, *Single-Source Shortest Paths*, especially Sections 4.4.1 and 4.4.3.